희소보상 및 장기계획 문제를 위한 계층적 강화학습 기반 Pokémon-Playing Agent



부산대학교 정보컴퓨터공학부

202155625 홍세민(semin73@pusan.ac.kr)

202055602 조재우(nicewodn@pusan.ac.kr)

지도교수: 김원석

목차

목차

1. 환경의 제약사항 및 기존 구조의 문제점 3

1.1. 포켓몬스터 제약사항 3

1.1.1. 기본 요구사항 3

1.1.2. 복합적 과제 3

1.1.3. 강화학습 사용 시 포켓몬스터 환경의 난관 4

1.2. 계층적 강화학습(기존 구조)기반 구조의 문제점 5

2. 문제 재정의 및 목표 7

2.1. 문제 재정의 7

2.2. 연구 수행방식 및 목표 7

2.2.1 연구 수행방식 7

2.2.2. 연구목표 7

3. 구조도 및 설계 구체화 8

3.1. 구조도 8

3.2. 설계 구체화 8

3.2.1. LLM 8

3.2.2. Dual MAP Architecture 9

3.2.3. RL 10

4. 과제 추진 계획 10

5. 구성원별 진척도 10

6. 과제 수행 내용 및 중간 결과 11

1. 환경의 제약사항 및 기존 구조의 문제점

1.1. 포켓몬스터 제약사항

판단능력을 지닌 사람은 포켓몬스터의 단순한 조작버튼 입력만으로 쉽게 게임을 진행할 수 있다. 하지만, 에이전트는 사람만큼의 판단능력이 없기에, 게임을 진행하려면 기본적인 버튼입력 외에도 다양한 요소를 고려하여 판단능력을 부여하는 과정이 필요하다.

1.1.1. 기본 요구사항

포켓몬스터 레드는 기본적인 조작버튼은 상·하·좌·우 방향키와 A, B 버튼 입력으로 구성된다. 에이전트는 이 조작버튼을 조합하여 전투, 이동, 상호작용 및 메뉴 선택 등의 복잡한 조작을 수행해야 한다.

1.1.2. 복합적 과제

이 게임을 해결하기 위해서는 기본적인 이동을 통해서 AI 에이전트가 다음과 같은 복합적인 과제들을 해결해야 한다.

- NPC 와의 대화 및 오브젝트와 상호작용
- 장거리 및 단거리 길 찾기 및 탐색
- 포켓몬 전투 및 포획

이러한 조건들은 단순한 정책만으로 해결하기 어려우며, 상황에 따른 의사결정과 장기적인 행동 계획을 요구한다.



그림 1 복합적인 과제로 이뤄진 포켓몬스터 레드 환경

1.1.3. 강화학습 사용 시 포켓몬스터 환경의 난관

[필수 스킬(비전머신) 사용 문제] 스토리 진행을 위해 장애물 혹은 퍼즐을 극복해야 하는데, 이를 위한 장애물 극복 스킬은 전체 게임에서 5 회 미만으로 사용된다. ⇒ 즉, 스킬의 중요도에 비해 환경에서 사용되는 횟수가 희소하여 상위 계층의 에이전트가 **적재적소에 스킬을 사용하도록 하는 학습 신호가 매우 적다**.

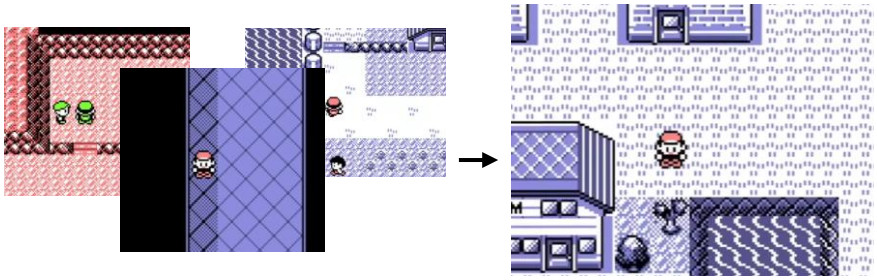


그림 2 학습을 통해 적재적소에 장애물 극복 스킬을 사용

[랜덤성 퍼즐 극복 문제] 스토리 진행을 위해 필수적으로 거쳐가야 하는 3 번째 체육관 관장에게 접근하려면 랜덤성에 의존한 퍼즐이 극복이 필요하다. ⇒ 일반적인 RL 에이전트는 한 번 오답이었던 곳을 탐색하지 않으려는 경향이 있다. 하지만 **오답이였던 선택지가 랜덤하게 다음 정답**이 될 수도 있기 때문에 시도를 강제할 특수한 수단이 필요하다.

[엄격한 가방 관리 문제] 포켓몬스터 레드는 모든 종류의 아이템을 합쳐 단 20 칸의 저장 공간만 제공한다. ⇒ 가방이 가득 차면 스토리 진행에 필수적인 아이템을 아예 획득하지 못할 가능성이 크며, 에이전트가 임의로 인벤토리를 비우다 **필수적인 아이템을 없애 버릴 위험이 존재**하여 고도화된 보상 설계가 필요하다.

[복잡한 다단계 선행 조건 문제 1] 스토리 진행을 위해 '플래시'라는 비전머신이 필요하며, 이를 얻으려면 도감 10 마리를 등록해야 한다. ⇒ 이를 달성하기 위해서 복잡한 "포켓볼 부족 → 돈 벌기 → 상점 이동 및 포켓볼 구매 → 원래 위치 복귀 및 포켓몬 포획" 이라는 고차원적 행동을 학습해야 한다.

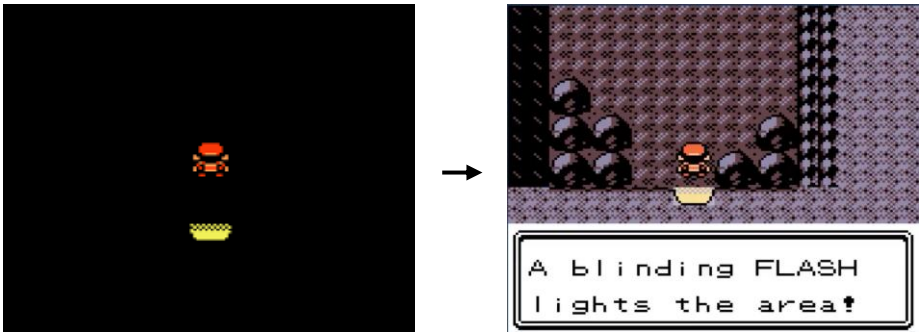


그림 3 돌산 터널에서 플래시를 사용

[복잡한 다단계 선행 조건 문제 2] 5 번째 도시 경비병 통과에서 4 번째 도시 백화점 6 층 자판기 '차가운 물'을 사서 전달해야 한다 ⇒ 백화점 1~5 층까지 수 많은 방해 요소가 존재하지만 소지금을 보존한 채 6 층 도달 후 '차가운 물'만 획득 후 사용도 하지 않아야 하는 고난도 제어가 요구된다.

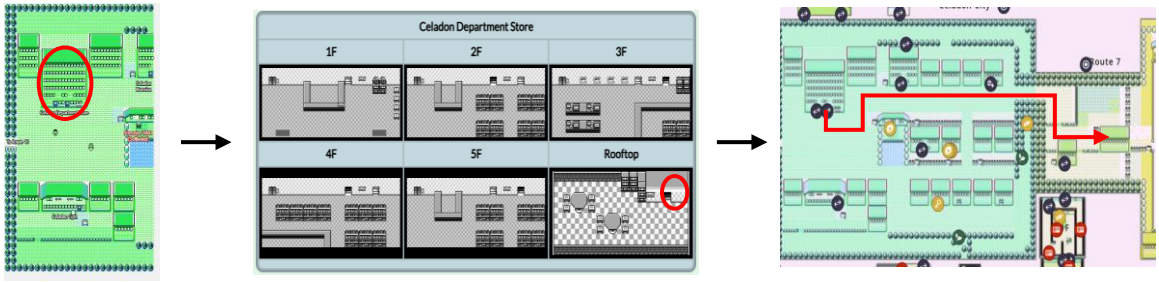


그림 4 적절한 위치로 이동하여 아이템 전달

즉, 두 경우 모두 스토리 진행을 위해, 게임에서 제공되는 힌트 혹은 자연스러운 보상 신호 없이 **특정한 구역에서 특정 행위를 정확하게 수행하는 것을 정확한 순서대로 진행**해야 스토리 진행이 가능하다.

1.2. 계층적 강화학습(기존 구조)기반 구조의 문제점

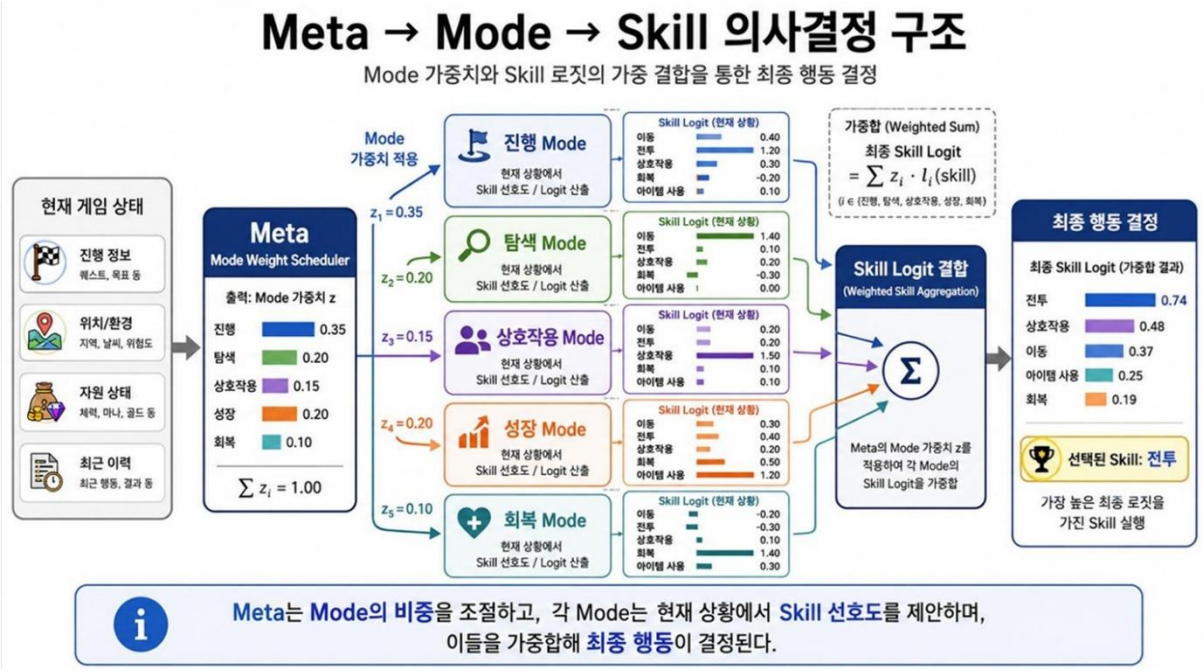


그림 5 기존 Meta-Controller 기반 3계층 아키텍처

기존 Meta-Controller 기반 3 계층 아키텍처는 아래와 같은 4 가지의 구조적 한계점이 존재한다.

1. Meta-Controller 역할의 한계

Meta-Controller는 RL로 구성되어, 현 게임상태를 RAM으로 읽고 각 Mode에 대한 가중치를 출력하는 일종의 **플래너**이다. 하지만, RL이 현재 이미지와 RAM 게임상태 정보를 읽는다 하더라도, 받은 정보에 대해서 잘 이해하고 판단하기에는 **state space가 너무 거대**하며, 이를 통한 Mode 가중치가 상황에 맞게 잘 적용될 수 있을 지 미지수이다. 또한 RL은 샘플 효율이 떨어져 이러한 아키텍처의 **학습 시간이 오래 걸릴 가능성**이 존재하고 학습이 잘 완료되더라도 **보상 해킹 같은 현상이 일어나 편향된 학습이 될 가능성**도 매우 크다.

2. Mode에 대한 과도한 역할 집중

Mode는 게임 진행에 필요한 새로운 지역 탐험, 마을 내에서 존재하는 오브젝트와 상호작용, 성장 등 **에이전트가 특정 행위의 집합을 지향**하도록 하는 에이전트이다. 하지만, Mode가 학습되는 과정에서 많은 문제를 한 번에 해결하도록 학습하는 과정에서 **역할이 과도하게 집중**되었다. 이로 인해 다양한 테스크를 성공적으로 수행할 수 있도록 하는 **학습의 난이도가 증가**한 것은 물론, 특정 mode에 게임 클리어를 위한 역할이 과도하게 배정되었다.

3. Skill 계층의 한계

Skill은 게임에서 반복적으로 등장하는 행위 혹은, 특정 퍼즐 극복을 위한 스킬 사용 등, 하나의 **성공과 실패가 명확한 task를 수행**하는 RL Agent이다. 하지만, 포켓몬 게임의 특성 상 게임 클리어를 위해 반드시 사용해야 하나 **단발성에 그치는 스킬들이 다수 존재**한다. 이런 Skill을 모두 RL로 학습하는 것이 합리적이지 않아, Skill에 대한 재구성 혹은 재정의가 필요하다.

4. 이벤트 선행조건 판단계층의 부재

포켓몬 이벤트 대부분 **선행조건이 존재**하지만 이를 **판단할 계층이 존재하지 않다**. 이는 에이전트가 학습에 있어 수많은 시행착오를 통해 해결하거나 운이 안 좋으면 방황하며 **게임 진행이 멈추고 학습이 수행되지 않을 가능성이 존재**한다.



그림 6 선행조건 판단 계층 부재 사례

결론적으로, 기존 3계층 아키텍처는 스토리의 흐름을 이해하는 '논리적 추론'이 불가능하다. 이를 해결하고자 에이전트에게 **명확한 방향성을 제시**하며 전체 **게임 맥락을 인지**하고 **단계적 목표를 설정**할 **LLM**을 도입하려고 한다.

2. 문제 재정의 및 목표

2.1. 문제 재정의

앞에서 제시했던 포켓몬스터 난관들은 여러가지 요소가 복합적으로 작용한 문제이다. 이를 바라보기 쉽게 하고 근본적인 문제부터 해결해 나가기 위해 문제를 재정의한다.

게임 진행에 모든 필수적 요소인 목표로의 이동을 가장 하위 계층 문제로 보고 상위 계층 문제는 다음에 의해 정의한다.

1. **현 계층의 문제의 해결이 전제가 되어 발생하는 문제**
2. **현 계층의 문제와 동시다발적으로 발생할 수 있는 문제**

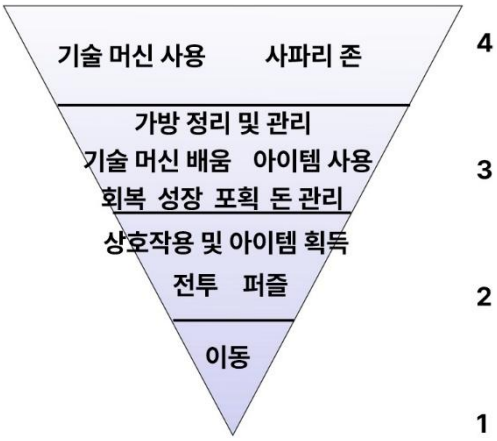


그림 7 문제 해결을 위한 계층적 모델

2.2. 연구 수행방식 및 목표

2.2.1 연구 수행방식

1. 계층적 관점에서의 점진적 문제 해결

문제의 재정의를 통해, 제시한 계층적인 관점에서 하위 계층으로부터 상위 계층으로 문제를 바라보고 하위 계층의 문제부터 **점진적으로 해결**한다. 이를 위해, **계층별 문제를 해결**할 수 있는 환경을 구성하여 개별 문제를 해결하고, 최종적으로는 **모든 문제들을 고려**한 포켓몬스터 클리어를 목표로 한다.

2. 보조자료 지도 도입

LLM 은 다양한 상황에서 응용을 할 수 있으나, 포켓몬스터 내부에서 목표를 제시하려면, **상황 인식에 대한 정보**가 필요하다. 이를 돕기 위해, 에이전트가 게임을 **진행하며 스스로 만들어가는 지도(Dual MAP)** 및 현재 상황에 대한 다양한 정보를 제공하여, LLM 이 적재적소에 하위 에이전트에게 목표를 제공할 수 있도록 돕는다.

2.2.2. 연구목표

1. LLM 플래너 도입을 통한 목표 지향적 파이프라인 구축

단일 RL 에이전트로는 포켓몬스터 내부의 목표 지향적인 이동을 달성하기 어렵다. 그렇기 때문에, **LLM 을 목표를 제시해주는 플래너로서 도입하여, 에이전트에게 인지 및 판단 능력을 부여하고 목표 지향적 이동을 달성한다.**

2. LLM 의존도 축소 및 자립형 에이전트 완성

성공적인 게임 플레이를 통해 데이터를 축적하며 이를 통해 **LLM 의 개입을 점차 줄이고 플래너 역할을 RL 이 대체**함으로, 최종적으로 **HRL 구조로 에이전트가 스스로 게임을 해결하는 것**을 목표로 한다.

3. 구조도 및 설계 구체화

3.1. 구조도

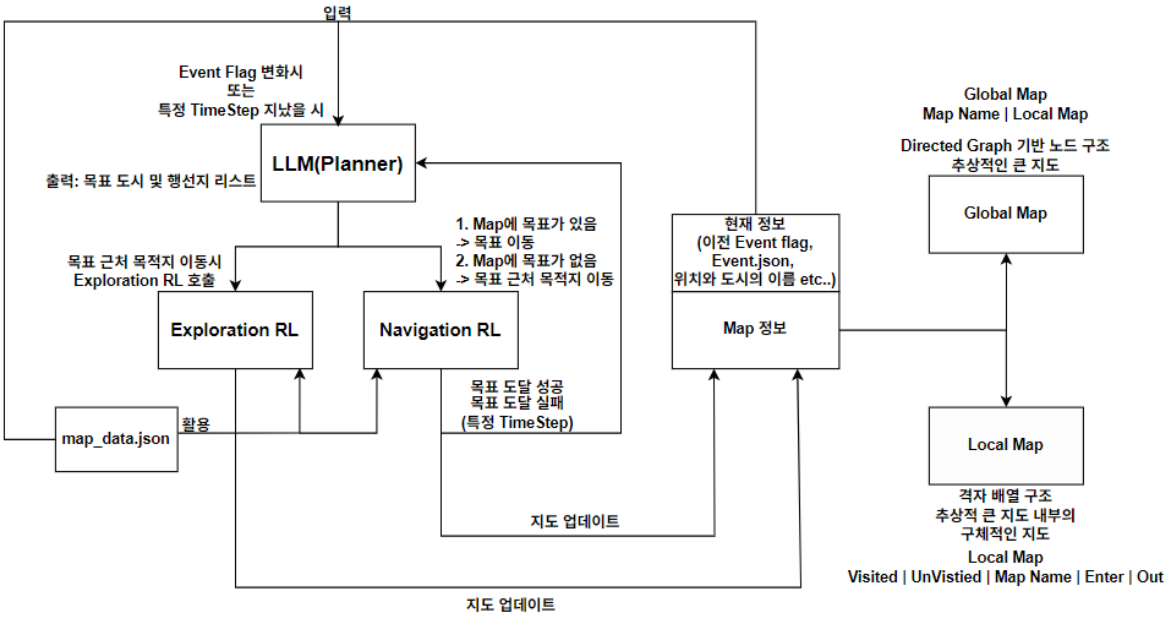


그림 8 LLM & MAP 구조도

전체적인 구조는 플래너의 역할을 하는 LLM, 지도를 확장하는 역할을 하는 Exploration RL, LLM 의 목표를 받아 이동하는 Navigation RL 과 현재 상태를 판단하는 기준이자 LLM 의 입력이 되는 RAM 정보와 지도(Global Map, Local Map)으로 구성된다.

3.2. 설계 구체화

3.2.1. LLM

LLM 은 플래너로서 RAM 정보와 지도를 기반으로 현재 상황을 판단하고 구체적인 목표를 제시한다.

- **호출 주기**: Event Flag 변화 시 또는 특정 Timestep 시
- **입력**: 현재상태(RAM 정보), Event Flag, Global Map
- **출력**: 목표 Map Name

3.2.2. Dual MAP Architecture

Map은 **(1)Global Map**과 **(2)Local Map**으로 구성되고, 에이전트가 새로운 환경에 대해서 탐험을 진행하며 **직접 지도를 그려나간다**.

■ **Global Map**

전체 월드맵 정보를 담고 있는 **전역적인 지도**이며, LLM이 **목표를 출력하기 위한 보조자료**이다. Global Map은 Directed Graph 형식으로 구현되어있다.

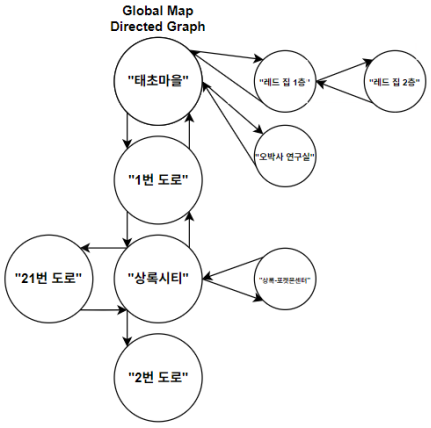


그림 9 Global Map 예시

■ Node와 Edge 구성

- **Node**
  - Map Name
  - Map_id
  - 해당 Local Map
- **Edge**
  - 단방향, 양방향 정보(0, 1)
  - Weight(1)

LLM이 목표를 제시했을 때, 현재 맵과 목표 맵을 기반으로 다익스트라, A*와 같은 **Navigation Algorithm**을 적용하여, 행선지 리스트를 뽑기 위해, 가중치를 각 1로 설정한다.

■ **Local Map**

Global Map 노드 속 내부 격자로 구성된 **구체적인 지도**이다. **NPC, Block 정보, Tile 정보를 표시**한다. 에이전트가 한 Step을 밟을 때마다, 20*18 타일의 화면 정보를 받아와 Local Map을 넓힌다.

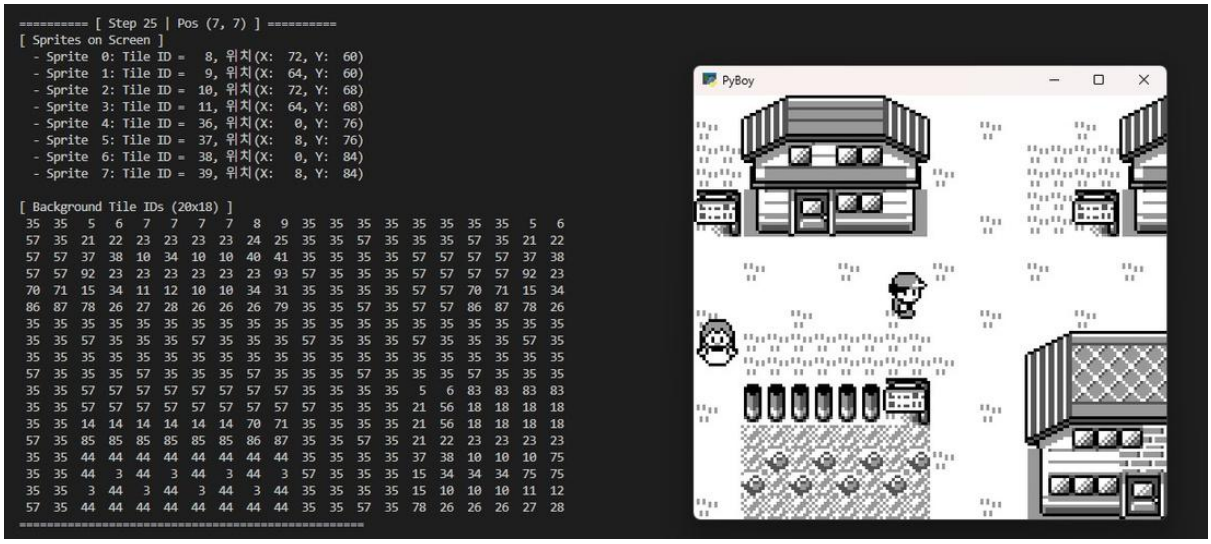


그림 10 현재 화면에 대한 Tile ID를 배열로 받은 화면

3.2.3. RL

단일 Skill 구조가 아닌 상황에 맞는 두 가지 에이전트로 구성된다.

- **Navigation RL**: **LLM 이 목표를 제시하면,** 해당 목표로 이동하는 RL 에이전트, 장애물을 회피하여 해당 **목적지까지의 최단 거리로 이동하는 것을 목적으로 한다.**

- **Exploration RL**: **목표에 대한 정보가 Map 에 존재하지 않을 때,** 미방문 타일을 중점으로 지도를 넓혀가며 목표를 발견할 때까지 **지도를 넓힌다**.

두 가지 에이전트를 활용하여 기존 맹목적 **호기심 기반 탐색이 아닌, 목표 지향적 아키텍처**를 설계한다.

4. 과제 추진 계획

구분	목표	5 월	6 월	7 월	8 월	9 월	10 월
환경 구축 및 아키텍처 설계	기존 아키텍처 한계점 도출						
	LLM & Dual Map 아키텍처 기획						
핵심 모듈 개발	Dual Map 시스템 구현						
	맵 확장 탐색 RL 및 이동 RL 학습						
파이프라인 연동 및 통합	LLM 프롬프트 최적화 및 API 연동						
	LLM 목표 지시 및 RL 행동 통합						
LLM 연동 및 최종 검증	성능 검증 및 파라미터 튜닝						
	데이터 축적 및 HRL 체계 연구						

5. 구성원별 진척도

이름	진척도
홍세민(팀장)	- LLM 플래너 & Dual Map 아키텍처 - 완료 - Dual Map 세부 구조 – 진행중 - Exploration RL 학습 – 진행중
조재우	- Navigation 알고리즘 구현 – 진행중 - LLM 프롬프트 최적화 및 API 연동 – 구상중

6. 과제 수행 내용 및 중간 결과

Dual Map 아키텍처 및 세부 구조 작성

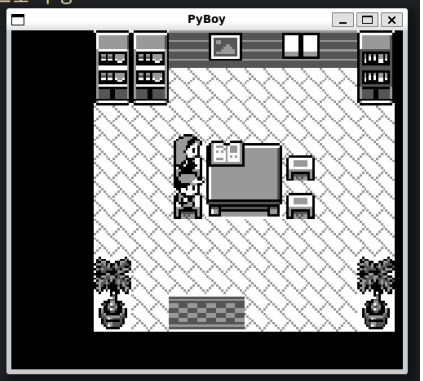
각 RL(Skill)들과 LLM 에게 넘길 정보를 구성하고, test_map.py 파일을 작성하여 기존의 Tile ID 로 받아온 정보를 2×2 캐릭터 이동 칸 단위인 블록으로 변경하고 각 셀의 정보를 Grid 로 실제 오류가 있는 지 검사한다.





그림 11 Dual Map Grids 검증 화면

```
       정답 타일 위치 후보: ['tiles[2,8] → 셀(4,1) 내부(0,0)', 'tiles[2,9] → 셀(4,1) 내부(1,0)', 'tiles[2,10] → 셀(5,1) 내부(0,0)'
, 'tiles[2,11] → 셀(5,1) 내부(1,0)', 'tiles[2,12] → 셀(6,1) 내부(0,0)', 'tiles[2,13] → 셀(6,1) 내부(1,0)', 'tiles[2,14] → 셀(7,1)
내부(0,0)', 'tiles[2,15] → 셀(7,1) 내부(1,0)']
       → 후보가 특정 셀로 일관되면 PLAYER_SCREEN_CELL_X/Y 또는 좌하단 인덱스를 그 값으로 수정
  [D] PASS 보이는 NPC 2개 모두 화면 범위 안 (-4 보정 정상)
  [E] PASS 스크롤 정렬됨 (SCX=0, SCY=208)
  [F] PASS 풀 타일: 게임=없음(0xFF), Map.py=없음(None)
  [F] 대화창 후보 관찰: wFontLoaded=0, wJoyIgnore=0x00 (대화 중일 때 값 변화 확인용)
  [i] slot  1: NPC picture=17 pos=(2,3)
  [i] slot  3: OBJECT picture=65 pos=(3,3)
  [i] GlobalMap: 노드 5개, 현재 맵 관측 210칸, 첫방문=True
  [M] 현재 장면 셀 매트릭스 (좌상단 셀 = 맵좌표 (-2,0))
      -2 -1  0  1  2  3  4  5  6  7
  y=  0 #  #  #  #  #  #  #  #  #
  y=  1 #  #  O  O  .  .  .  .  O
  y=  2 #  #  .  .  .  .  .  .  .
  y=  3 #  #  .  .  N  O  #  .  .
  y=  4 #  #  .  .  P  #  #  .  .
  y=  5 #  #  .  .  o  .  .  .  .
  y=  6 #  #  #  .  o  .  .  .  #
  y=  7 #  #  #  .  o  .  .  .  #
  y=  8 #  #  #  #  #  #  #  #  #
      (P=플레이어  .=통행  o=통행(방문)  #=벽  ~=물  G=전투풀  U/D/L/R=언덕  T=풀베기나무  N=NPC  O=오브젝트  (공백)=미관측)
  [M] to_matrix('block') 화면 영역 (-1=미관측 0=통행 1=벽 2=물 3=전투풀 4=U 5=D 6=L 7=R):
       1  1  1  1  1  1  1  1  1  1
       1  1  1  1  0  0  0  0  0  1
       1  1  0  0  0  0  0  0  0  0
       1  1  0  0  0  1  1  0  0  0
       1  1  0  0  0  1  1  0  0  0
       1  1  0  0  0  0  0  0  0  0
       1  1  1  0  0  0  0  0  0  1
       1  1  1  0  0  0  0  0  0  1
       1  1  1  1  1  1  1  1  1  1
```



지도 요약을 통해 Directed Graph 로 구성된 GlobalMap 을 확인하고 LocalMap 에 존재하는 POI 정보를 보인다.

```
============================================================
누적 리포트
============================================================
  [B] walkable 리스트 대조: PASS 220 / FAIL 0  → 검증 완료 ✓
  [C] 플레이어 셀/좌하단 타일: PASS 123 / FAIL 97  → 실패 있음 ✗
  [D] 스프라이트 좌표 오프셋: PASS 118 / FAIL 0  → 검증 완료 ✓
  [E] 스크롤 정렬: PASS 220 / FAIL 0  → 검증 완료 ✓

지도 요약:
- Pallet Town (id=0): 관측 465칸, 출구→[Players House First Floor, Route 1, Rivals House, Oaks Lab], POI=[Oaks Lab, Players House F
irst Floor, Rivals House, Route 1]
- Players House First Floor (id=37): 관측 244칸, 출구→[Pallet Town, Players House Second Floor], POI=[Pallet Town, Players House Se
cond Floor]
- Players House Second Floor (id=38): 관측 153칸, 출구→[Players House First Floor], POI=[Players House First Floor]
- Route 1 (id=12): 관측 140칸, 출구→[Pallet Town], POI=[Pallet Town]
- Rivals House (id=39): 관측 210칸, 출구→[Pallet Town], POI=[Pallet Town]
- Oaks Lab (id=40): 관측 319칸, 출구→[Pallet Town], POI=[Pallet Town]
```

그림 12 Dual Map Global Map Node 및 Cell_Name 검증

Exploration Skill

다음과 같은 보상 항목과 기준을 통해 Exploration Skill 학습을 진행한다.

- **보상 항목**

보상	트리거	기본값	가중치	실제 지급량
new_tile	에피소드 내 처음 밟은 타일(전투·SKIP 맵 제외)	new_tile_base =0.008	1.0	0.008 X tile_scale X 1.0
new_map	에피소드 내 처음 방문한 맵 ID(시작 맵·전투·SKIP 제외)	new_map_base =2.0	1.0	2.0 X 1.0 = 2.0

- **Map_id 별 new_tile 체류 시간 감쇠 (tile_scale)**

맵 카테고리	설명	Decay_steps	감쇠 공식
Hub	마을·도시	1000	Max(0.15, 1 − 체류스텝 / decay_steps)
Hub_facility	포켓센터·체육관	800	↑
Transit	길·루트	1500	↑
Cave_like	동굴·탑·던전	1500	↑
interior	집·실내	500	↑

Navigation Skill

현재까지 포켓몬 레드 환경의 타일셋, 블록 정보, collision tile, warp tile, ledge tile 정보를 분석하여 각 맵 이동 가능 여부가 포함된 Local Map 및 다음 맵으로 가는 정보와 warp 좌표를 가진 Node 를 통해 생성된 Global Map 을 Navigation 용 Map 을 생성하였다.

위 맵 데이터를 이용해 Global Map 을 통해서 최종 목적지까지의 이동 경로를 다익스트라 알고리즘으로 계산하고, 계산한 경로를 Queue 형태로 관리하여 Local Map 내부에서 A* 알고리즘을 사용하여 최종목적지까지의 이동 경로를 이동하는 Navigation 모듈을 구현할 예정이다. 최종적으로 실제 조작 명령으로 변환하여 PyBoy 환경에서 자동 이동을 수행하도록 확장할 계획이다.

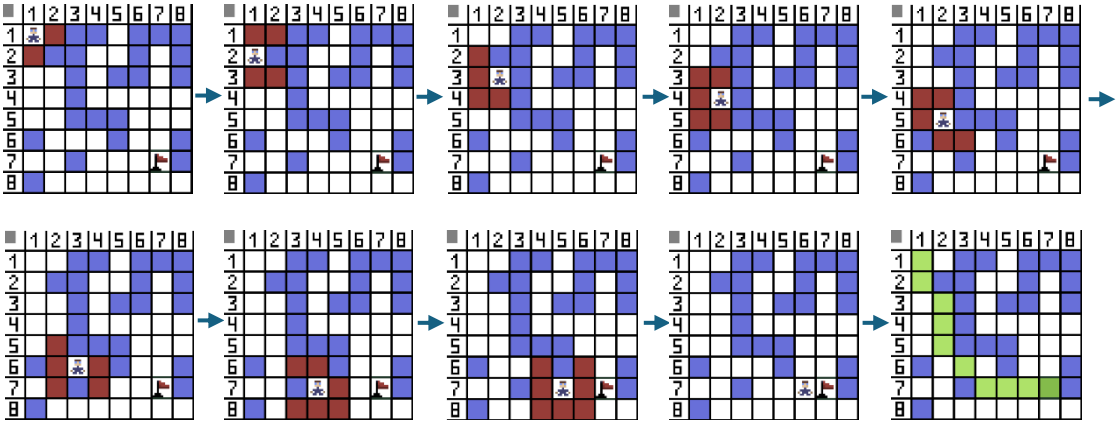


그림 13 A* 알고리즘 적용 사례